

AnalisePGAdmin

2025-11-29

1. Conectando-se ao BD

```
DB_HOST <- "localhost"
DB_PORT <- 5432
DB_NAME <- "Teste"
DB_USER <- "postgres"
DB_PASS <- "1234"

# --- Estabelecer a Conexão ---
con <- dbConnect(
  RPostgres::Postgres(),
  host = DB_HOST,
  port = DB_PORT,
  dbname = DB_NAME,
  user = DB_USER,
  password = DB_PASS
)

# Para verificar se a conexão foi bem-sucedida
print(con)
```

```
## <PqConnection> Teste@localhost:5432
```

2. Obtenção dos dados

```
# Obtenção de dados da tabela 'participa_em' (Matrículas)
matriculas <- dbReadTable(con, "participa_em")

# Criação de chave primária composta (como no slide)
matriculas$key <- paste(
  matriculas$nusp_aluno,
  matriculas$vigencia,
  matriculas$sigla_disciplina,
  matriculas$codigo_turma,
  sep = ""
)
row.names(matriculas) <- matriculas$key

df <- matriculas # Usando 'df' como o dataframe principal de matrículas
```

```
# Obtenção de dados das tabelas 'sala' e 'instituto'
salas <- dbReadTable(con, "sala")
institutos <- dbReadTable(con, "instituto")
```

3. Análise Gráfica

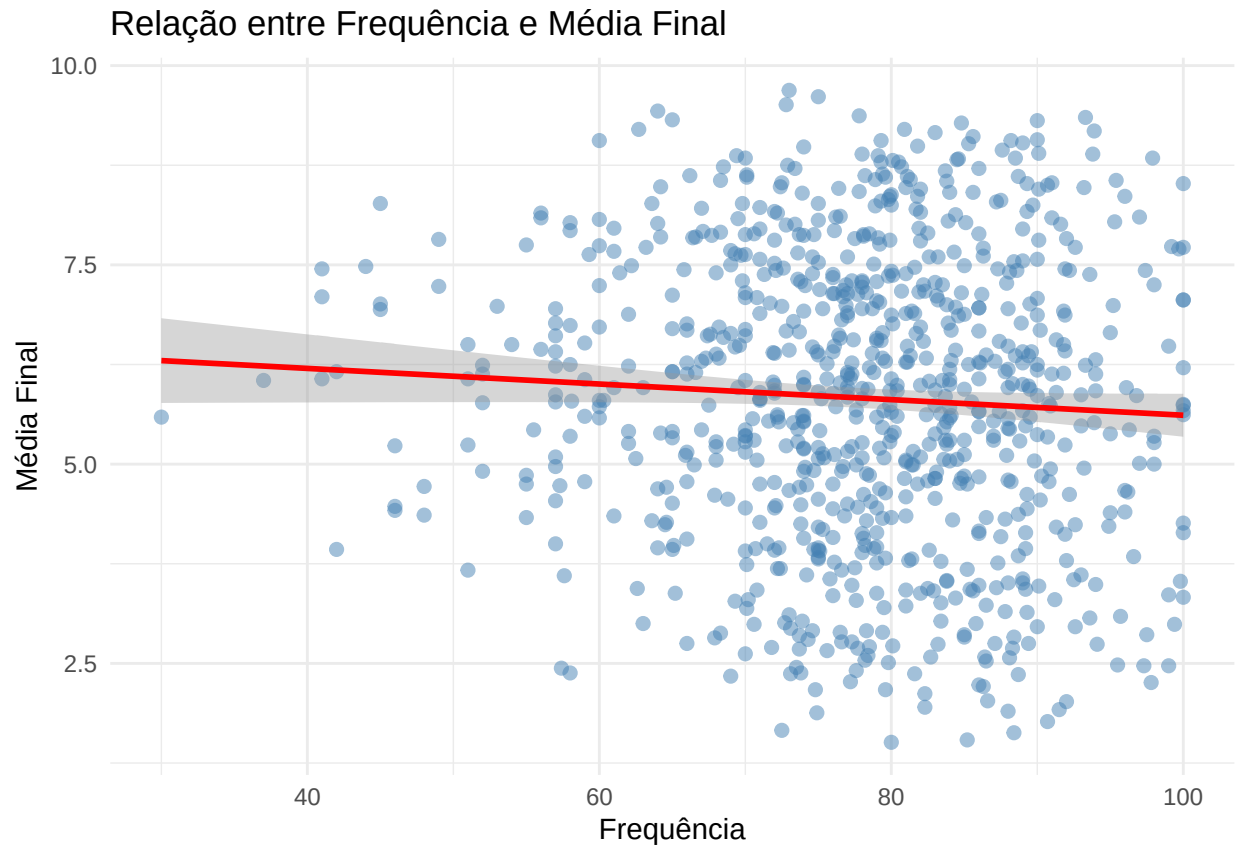
Matrículas

```
library(ggplot2)
df <- matriculas
ggplot(df, aes(x = frequencia, y = media_final)) +
  geom_point(alpha = 0.5, color = "steelblue", size = 2) +
  geom_smooth(method = "lm", color = "red", se = TRUE) + # Adiciona linha de regressão
  labs(
    title = "Relação entre Frequência e Média Final",
    x = "Frequência",
    y = "Média Final"
  ) +
  theme_minimal()
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

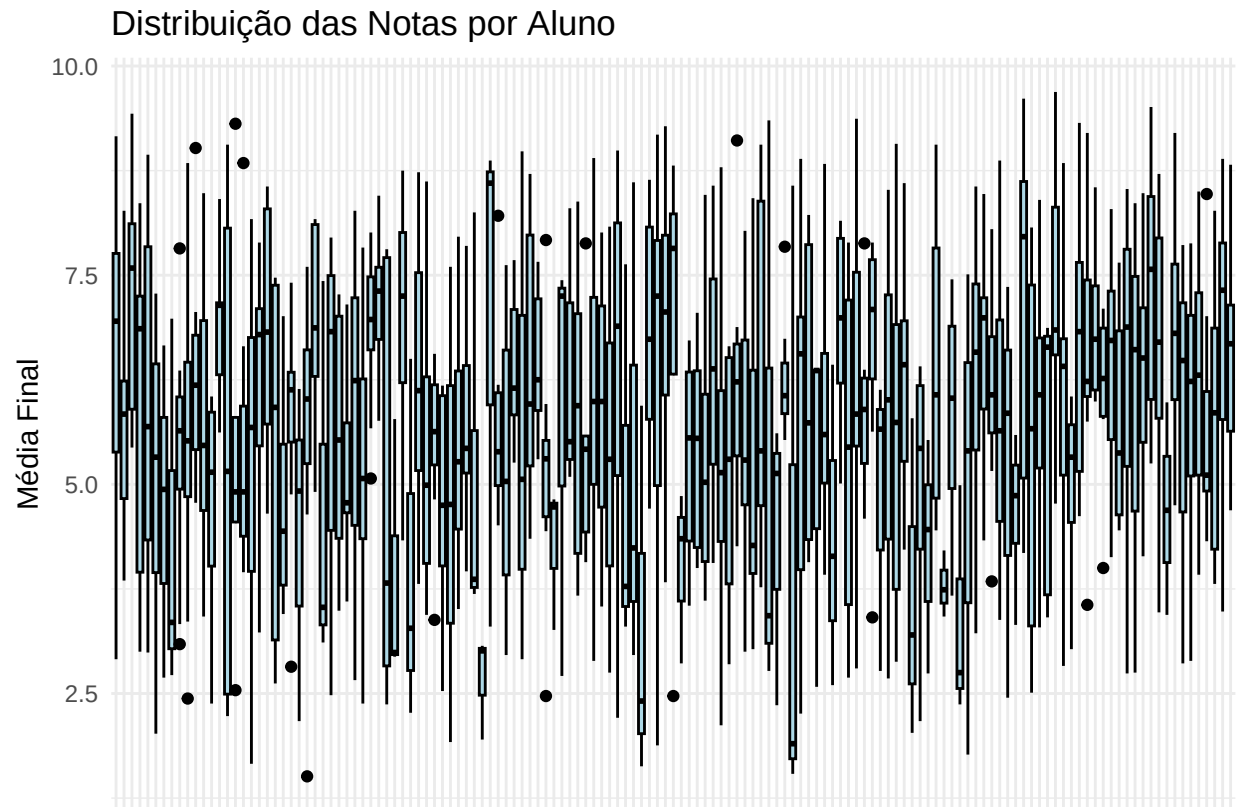
```
## Warning: Removed 157 rows containing non-finite outside the scale range
## ('stat_smooth()').
```

```
## Warning: Removed 157 rows containing missing values or values outside the scale range
## ('geom_point()').
```



```
ggplot(matriculas, aes(x = factor(nusp_aluno), y = media_final)) +
  geom_boxplot(fill = "lightblue", color = "black") +
  labs(
    title = "Distribuição das Notas por Aluno",
    x = "", # Eixo X sem rótulo para não poluir (como no slide)
    y = "Média Final"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_blank(), # Remove rótulos NUSP
    axis.ticks.x = element_blank() # Remove as marcas do eixo X
  )
```

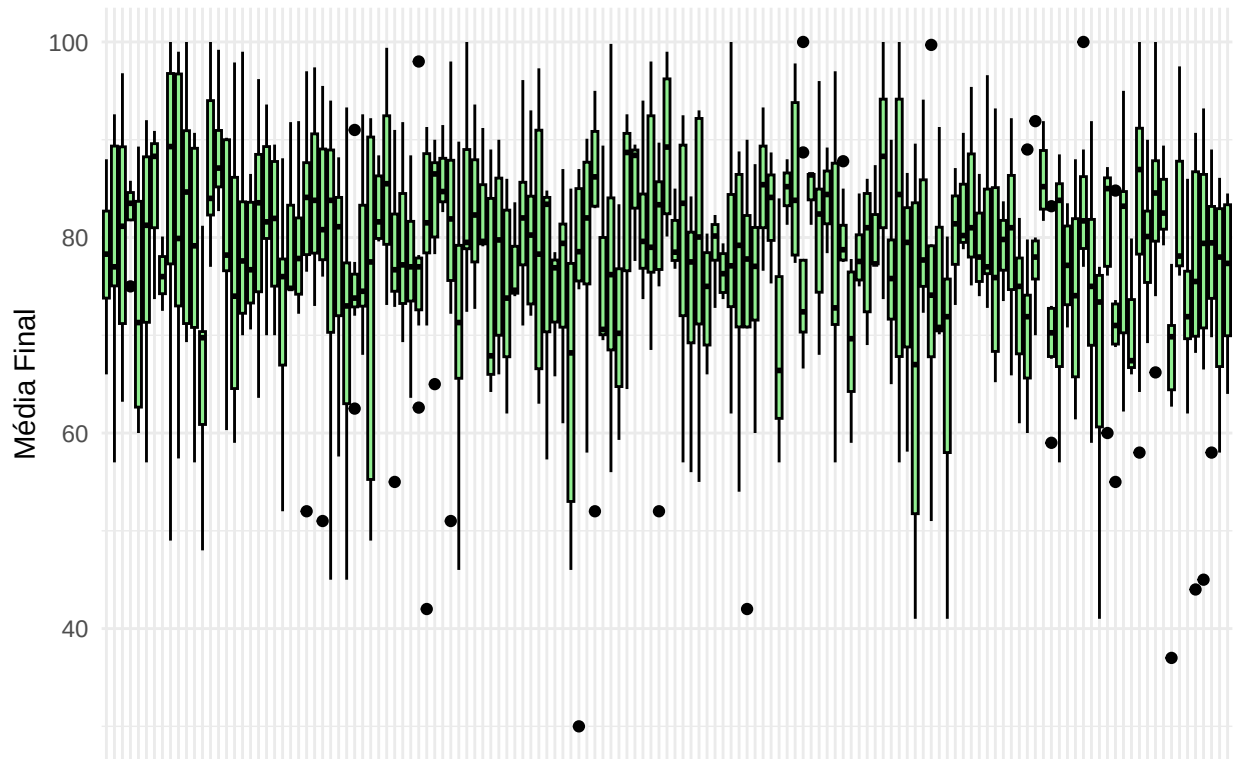
```
## Warning: Removed 157 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```



```
ggplot(matriculas, aes(x = factor(nusp_aluno), y = frequencia)) +
  geom_boxplot(fill = "lightgreen", color = "black") +
  labs(
    title = "Distribuição das Notas por Aluno",
    x = "", # Eixo X sem rótulo para não poluir (como no slide)
    y = "Média Final"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_blank(), # Remove rótulos NUSP
    axis.ticks.x = element_blank() # Remove as marcas do eixo X
  )
```

```
## Warning: Removed 157 rows containing non-finite outside the scale range
## ('stat_boxplot()').
```

Distribuição das Notas por Aluno



```
library(dplyr)
library(ggplot2)

# Código R: Filtra NAs antes da plotagem
df_plot_notas_validas <- df %>%
  mutate(
    Semestre = case_when(
      grepl("-1", vigencia) ~ "Primeiro Semestre",
      grepl("-2", vigencia) ~ "Segundo Semestre",
      TRUE ~ "Outro"
    )
  ) %>%
  # Passo crucial: Remove linhas onde a média final não está preenchida
  filter(!is.na(media_final)) %>%
  filter(Semestre %in% c("Primeiro Semestre", "Segundo Semestre"))

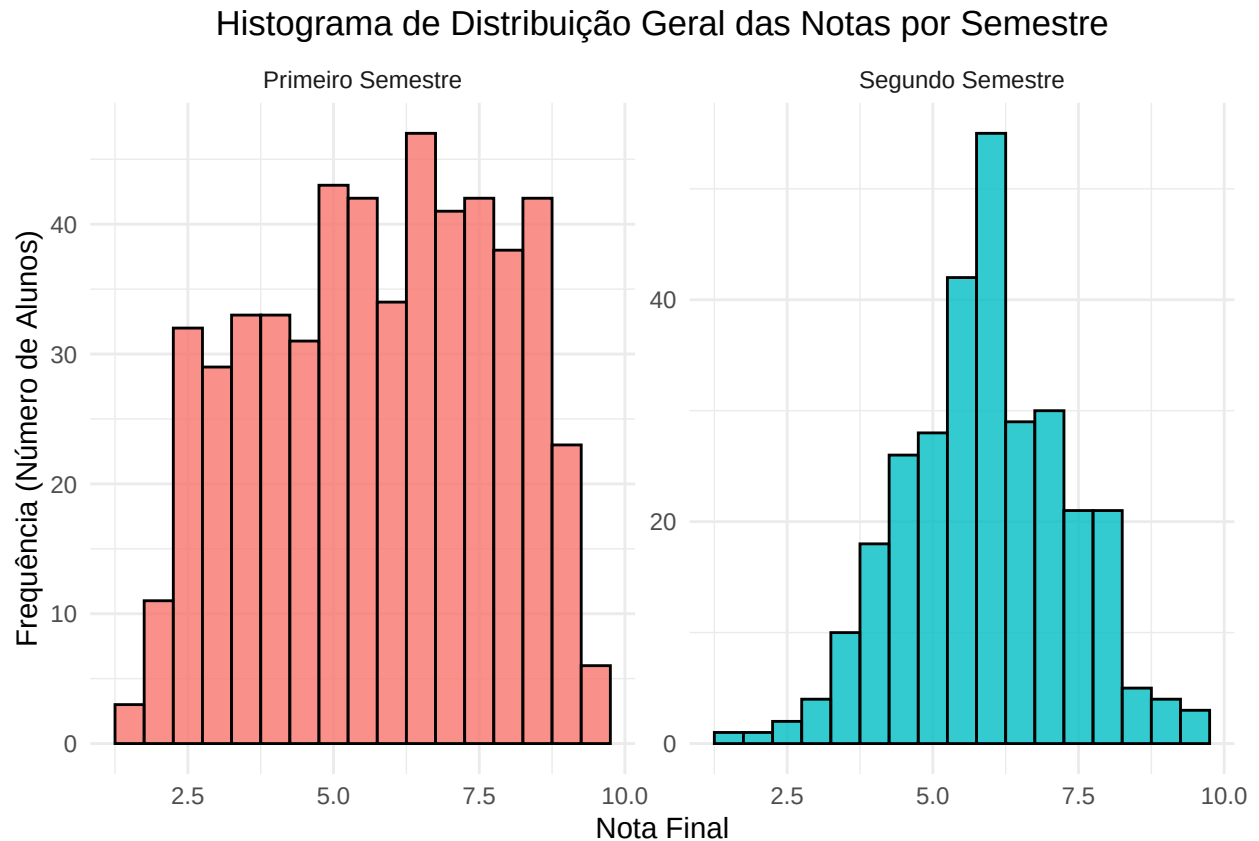
# Checagem final (garante que há dados válidos em algum dos grupos)
if (nrow(df_plot_notas_validas) > 0) {
  ggplot(df_plot_notas_validas, aes(x = media_final, fill = Semestre)) +
    geom_histogram(binwidth = 0.5, color = "black", alpha = 0.8) +
    facet_wrap(~ Semestre, scales = "free_y") +
    labs(
      title = "Histograma de Distribuição Geral das Notas por Semestre",
      x = "Nota Final",
      y = "Frequência (Número de Alunos)"
    ) +

```

```

    theme_minimal() +
    theme(legend.position = "none", plot.title = element_text(hjust = 0.5))
  } else {
  }
}

```



Cursos com maior quantidade de alunos

```

#Utilizando comando SQL para contar número de alunos por curso
sql_alunos_curso <- "
  SELECT c.nome AS nome_curso, COUNT(a.nusp_usuario) AS total_alunos
  FROM aluno AS a
  JOIN curso AS c ON a.id_curso = c.id_curso
  GROUP BY c.nome
  ORDER BY total_alunos DESC
  LIMIT 5;
"
df_alunos_curso <- dbGetQuery(con, sql_alunos_curso)

ggplot(df_alunos_curso, aes(x = reorder(nome_curso, total_alunos), y = total_alunos, fill = nome_curso)) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = total_alunos), hjust = -0.5) +
  coord_flip() +
  labs(

```

```

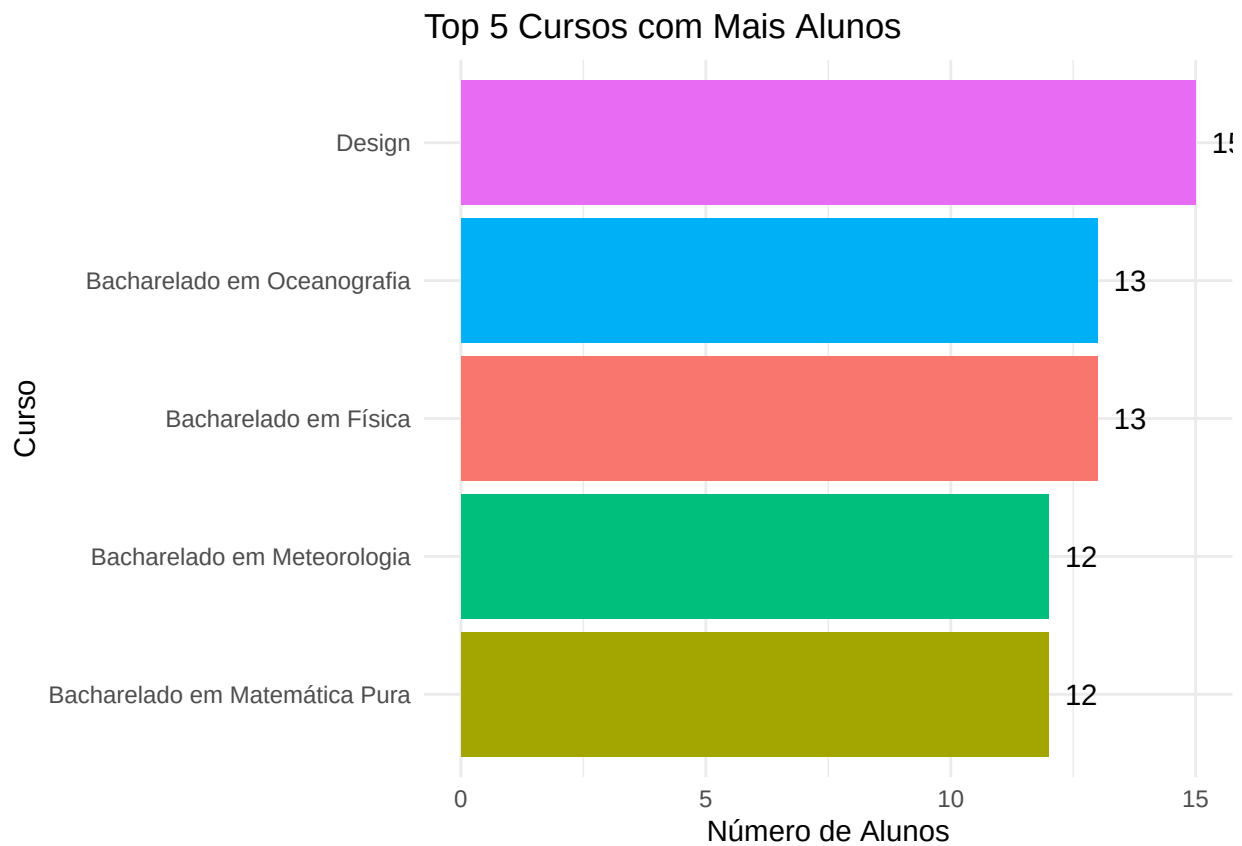
    title = "Top 5 Cursos com Mais Alunos",
    x = "Curso",
    y = "Número de Alunos"
) +
theme_minimal()

```

```

## Don't know how to automatically pick scale for object of type <integer64>.
## Defaulting to continuous.
## Don't know how to automatically pick scale for object of type <integer64>.
## Defaulting to continuous.

```



Disciplinas oferecidas por vigência

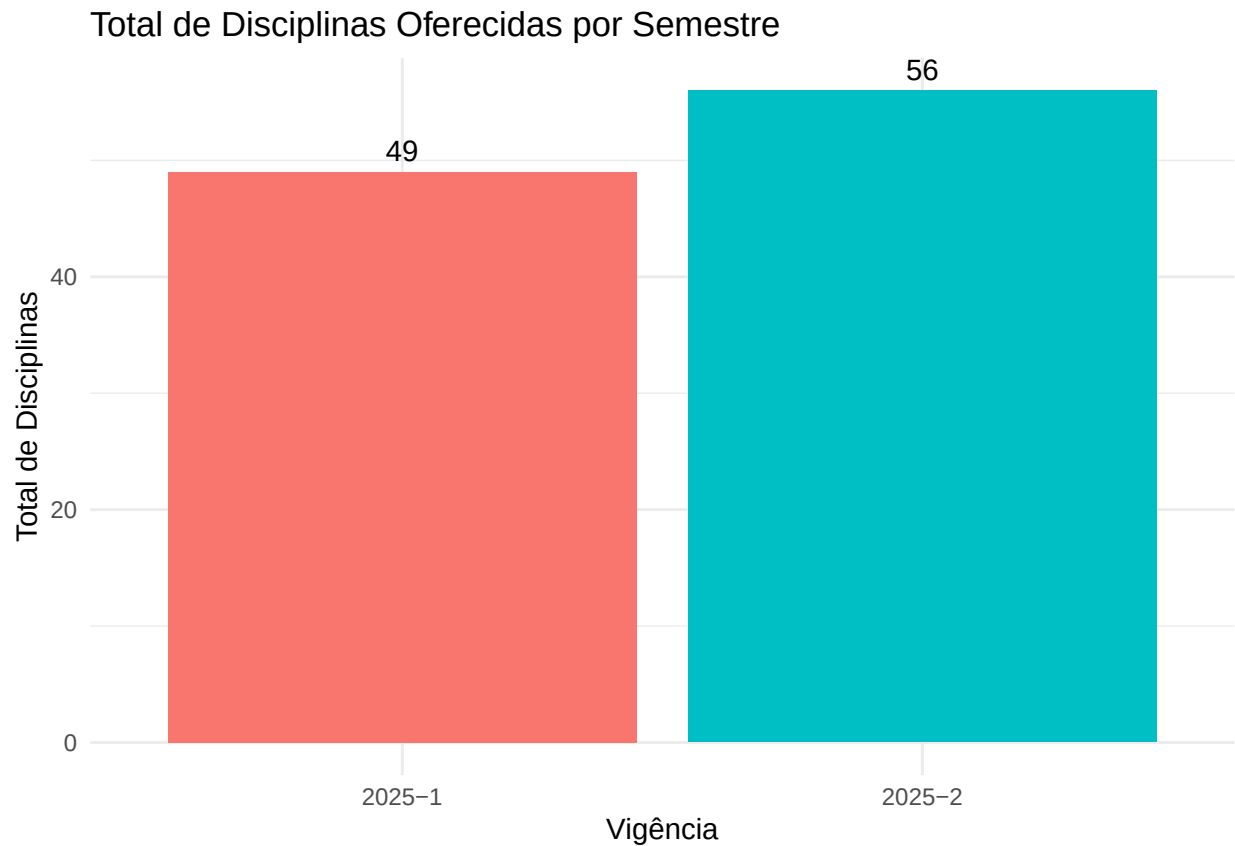
```

sql_oferta_vigencia <- "
  SELECT
    vigencia,
    COUNT(DISTINCT sigla_disciplina) AS total_disciplinas
  FROM
    turma
  GROUP BY
    vigencia
  ORDER BY
    vigencia;
"
df_oferta_vigencia <- dbGetQuery(con, sql_oferta_vigencia)

```

```
ggplot(df_oferta_vigencia, aes(x = vigencia, y = total_disciplinas, fill = vigencia)) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = total_disciplinas), vjust = -0.5) +
  labs(title = "Total de Disciplinas Oferecidas por Semestre", x = "Vigência", y = "Total de Disciplinas") +
  theme_minimal()
```

```
## Don't know how to automatically pick scale for object of type <integer64>.
## Defaulting to continuous.
## Don't know how to automatically pick scale for object of type <integer64>.
## Defaulting to continuous.
```



Proporções de pedidos de reserva

```
sql_status_reserva <- "
SELECT
  CASE
    WHEN status = 1 THEN '1 - Pendente'
    WHEN status = 2 THEN '2 - Aceito'
    WHEN status = 3 THEN '3 - Negado'
  END AS status_label,
  COUNT(id_pedido) AS total
FROM
```



```

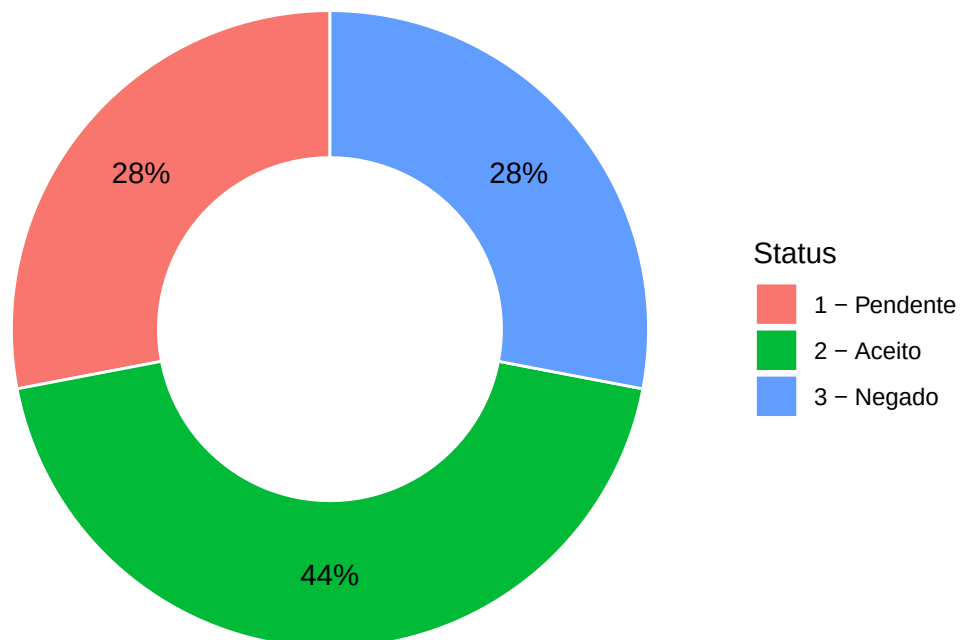
    pedido_reserva
  GROUP BY
    status_label
  ORDER BY
    status_label;
"
df_status_reserva <- dbGetQuery(con, sql_status_reserva)

df_status_reserva <- df_status_reserva %>%
  mutate(
    porcentagem = total / sum(total),
    label_pos = cumsum(porcentagem) - (0.5 * porcentagem),
    label = paste0(round(porcentagem * 100), "%")
  )

ggplot(df_status_reserva, aes(x = 2, y = porcentagem, fill = status_label)) +
  geom_bar(stat = "identity", color = "white") +
  coord_polar(theta = "y", start = 0) +
  geom_text(aes(y = label_pos, label = label), color = "black", size = 4) +
  xlim(0.5, 2.5) +
  labs(title = "Proporção de Status dos Pedidos de Reserva", fill = "Status") +
  theme_void() +
  theme(plot.title = element_text(hjust = 0.5))

```

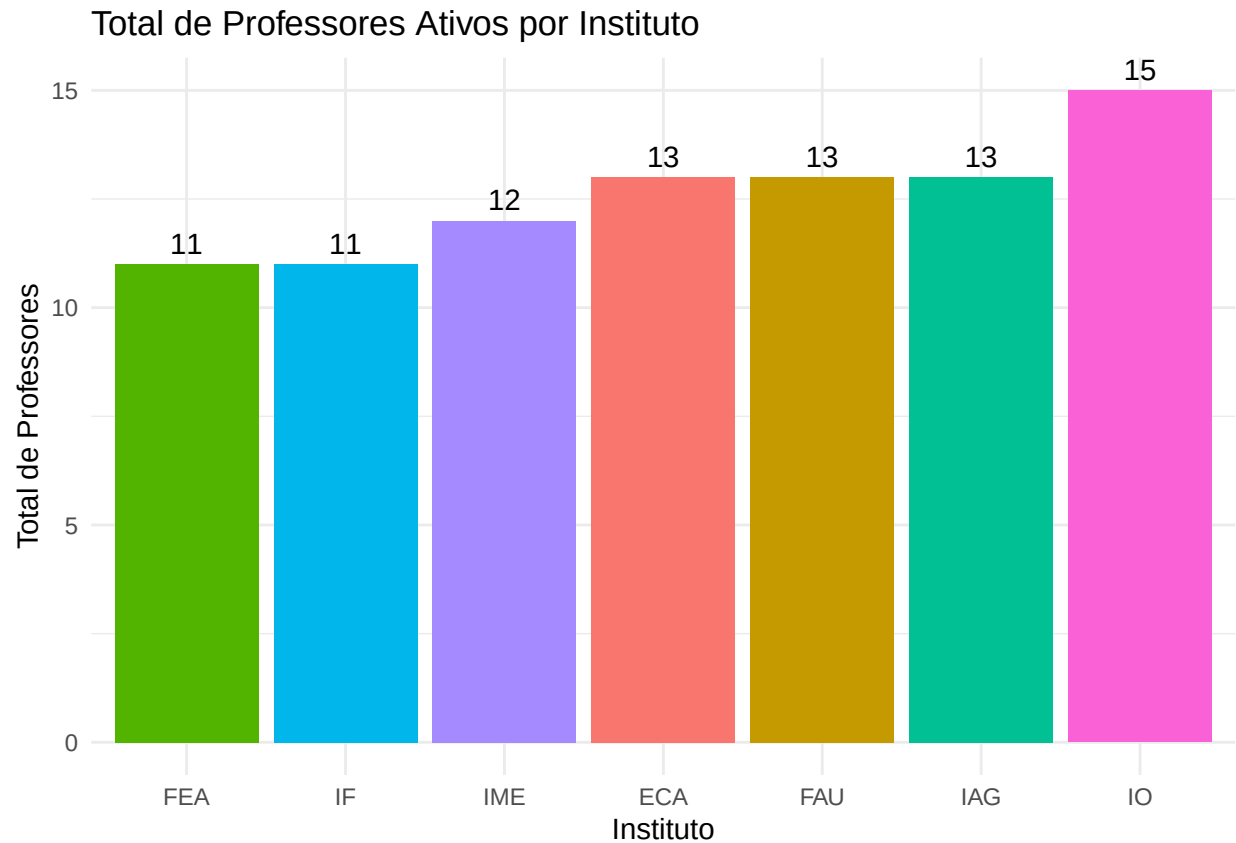
Proporção de Status dos Pedidos de Reserva



Professores ativos por institutos

```
sql_professores_instituto <- "  
  SELECT  
    i.sigla AS sigla_instituto,  
    COUNT(vt.nusp_associado) AS total_professores  
  FROM  
    vinculo_trabalho AS vt  
  JOIN  
    instituto AS i ON vt.id_instituto = i.id_instituto  
  WHERE  
    vt.cargo LIKE 'Professor %' AND vt.data_fim IS NULL -- Apenas professores ativos  
  GROUP BY  
    i.sigla  
  ORDER BY  
    total_professores DESC;  
"  
df_prof_instituto <- dbGetQuery(con, sql_professores_instituto)  
  
ggplot(df_prof_instituto, aes(x = reorder(sigla_instituto, total_professores), y = total_professores, f  
  geom_col(show.legend = FALSE) +  
  geom_text(aes(label = total_professores), vjust = -0.5) +  
  labs(title = "Total de Professores Ativos por Instituto", x = "Instituto", y = "Total de Professores  
  theme_minimal()
```

```
## Don't know how to automatically pick scale for object of type <integer64>.  
## Defaulting to continuous.  
## Don't know how to automatically pick scale for object of type <integer64>.  
## Defaulting to continuous.
```

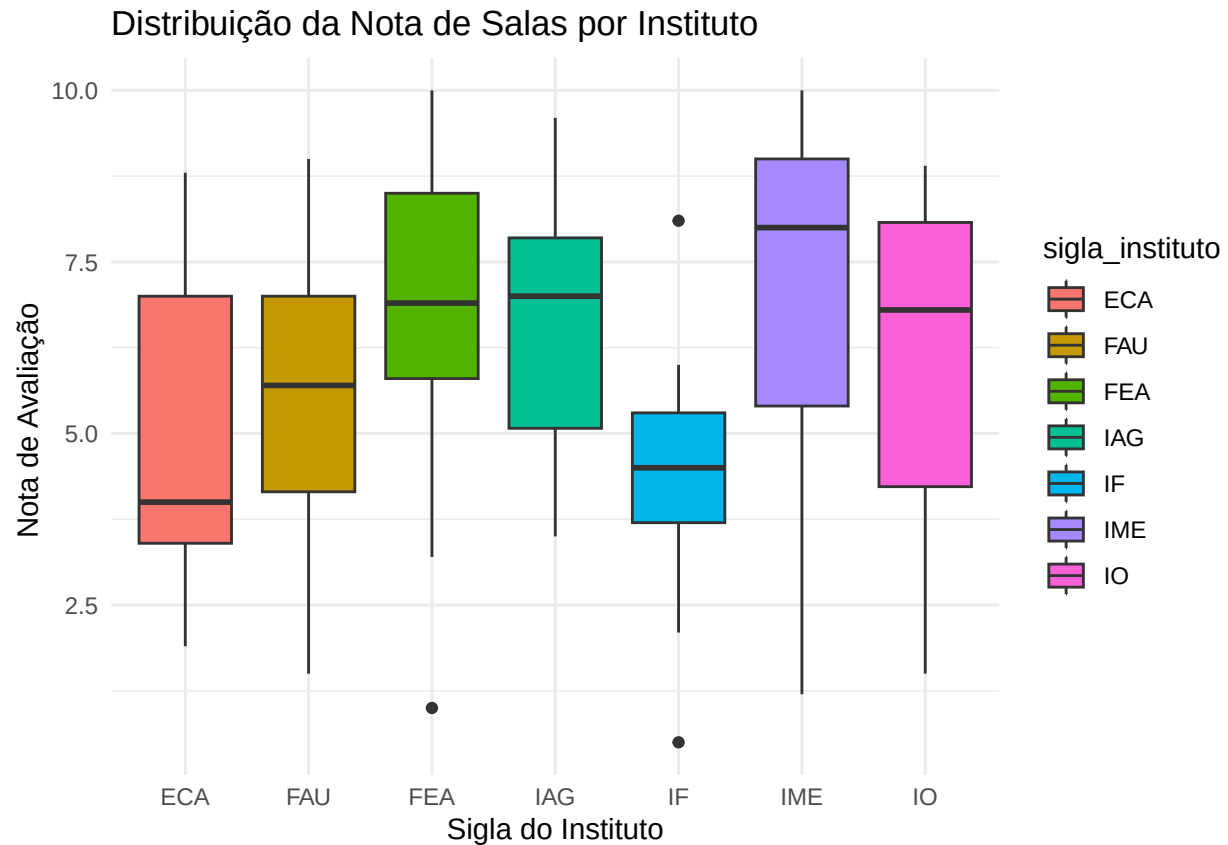


Análise de Salas e avaliações

```
#Comando SQL englobado em R para fazer consultas mais fácil
sql_avaliacao_sala <- "
  SELECT
    a.nota,
    i.sigla AS sigla_instituto
  FROM
    avaliacao_sala AS a
  JOIN
    instituto AS i ON a.id_instituto = i.id_instituto;
"

#Avaliações por salas por instituo. (Buscamos encontrar os institutos com as "melhores salas")
df_avaliacao_sala <- dbGetQuery(con, sql_avaliacao_sala)

ggplot(df_avaliacao_sala, aes(x = sigla_instituto, y = nota, fill = sigla_instituto)) +
  geom_boxplot() +
  labs(
    title = "Distribuição da Nota de Salas por Instituto",
    x = "Sigla do Instituto",
    y = "Nota de Avaliação "
  ) +
  theme_minimal()
```

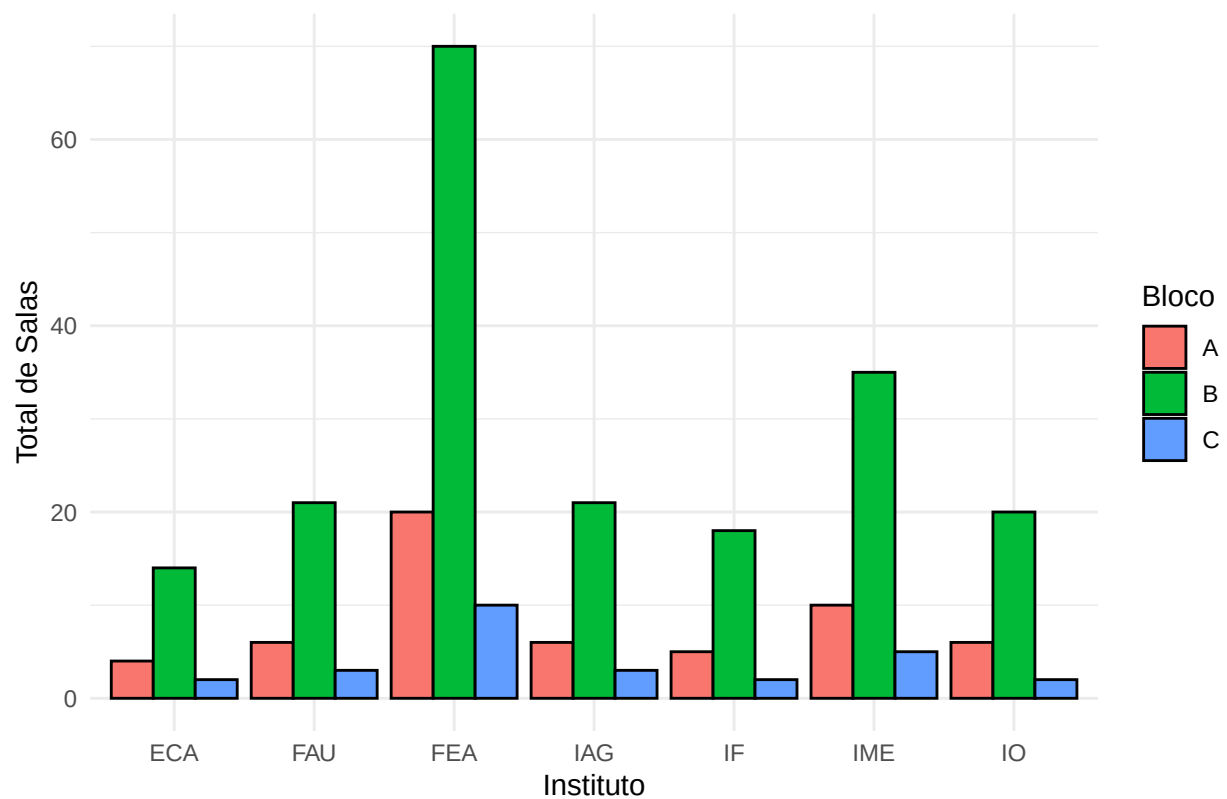


```
# Números de salas por blocos em institutos
sql_salas_bloco_inst <- "
SELECT
  i.sigla AS sigla_instituto,
  SUBSTRING(s.id_sala FROM 1 FOR 1) AS bloco,
  COUNT(s.id_sala) AS total_salas
FROM
  sala AS s
JOIN
  instituto AS i ON s.id_instituto = i.id_instituto
GROUP BY
  i.sigla, bloco
ORDER BY
  i.sigla, bloco;
"
df_salas_bloco_inst <- dbGetQuery(con, sql_salas_bloco_inst)

ggplot(df_salas_bloco_inst, aes(x = sigla_instituto, y = total_salas, fill = bloco)) +
  geom_bar(stat = "identity", position = position_dodge(width = 0.9), color = "black") +
  labs(title = "Contagem de Salas por Bloco e Instituto", x = "Instituto", y = "Total de Salas", fill = "bloco") +
  theme_minimal()
```

```
## Don't know how to automatically pick scale for object of type <integer64>.
## Defaulting to continuous.
```

Contagem de Salas por Bloco e Instituto



#Distribuição da lotação máxima de salas (Esperado - Dados uniformes)

```
sql_lotacao_maxima <- "SELECT lot_max FROM sala;"
```

```
df_lotacao <- dbGetQuery(con, sql_lotacao_maxima)
```

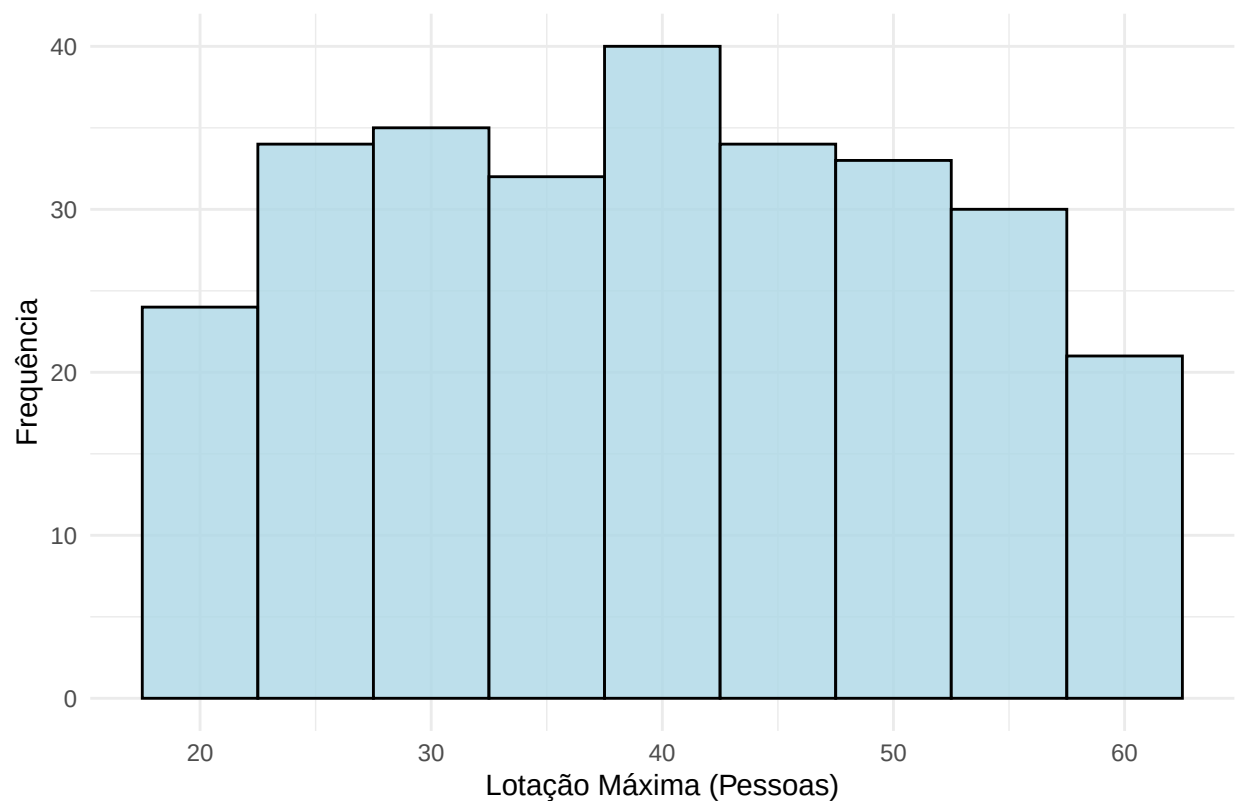
```
ggplot(df_lotacao, aes(x = lot_max)) +
```

```
  geom_histogram(binwidth = 5, fill = "lightblue", color = "black", alpha = 0.8) +
```

```
  labs(title = "Distribuição da Lotação Máxima das Salas", x = "Lotação Máxima (Pessoas)", y = "Frequência")
```

```
  theme_minimal()
```

Distribuição da Lotação Máxima das Salas



Disciplinas e departamentos

#Utilizando nossas visões para facilitar o alcance dos dados

```
sql_disciplinas_depto <- "
```

```
  SELECT
    nome_departamento,
    COUNT(sigla_disciplina) AS total_disciplinas
  FROM
    View_Disciplinas_Departamento
  GROUP BY
    nome_departamento
  ORDER BY
    total_disciplinas DESC;
```

```
"
```

```
df_disciplinas_depto <- dbGetQuery(con, sql_disciplinas_depto)
```

```
ggplot(df_disciplinas_depto, aes(x = reorder(nome_departamento, total_disciplinas), y = total_disciplinas)) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = total_disciplinas), hjust = -0.5, size = 3.5) +
  coord_flip() +
  labs(
    title = "Oferta de Disciplinas por Departamento",
    x = "Departamento",
```

```

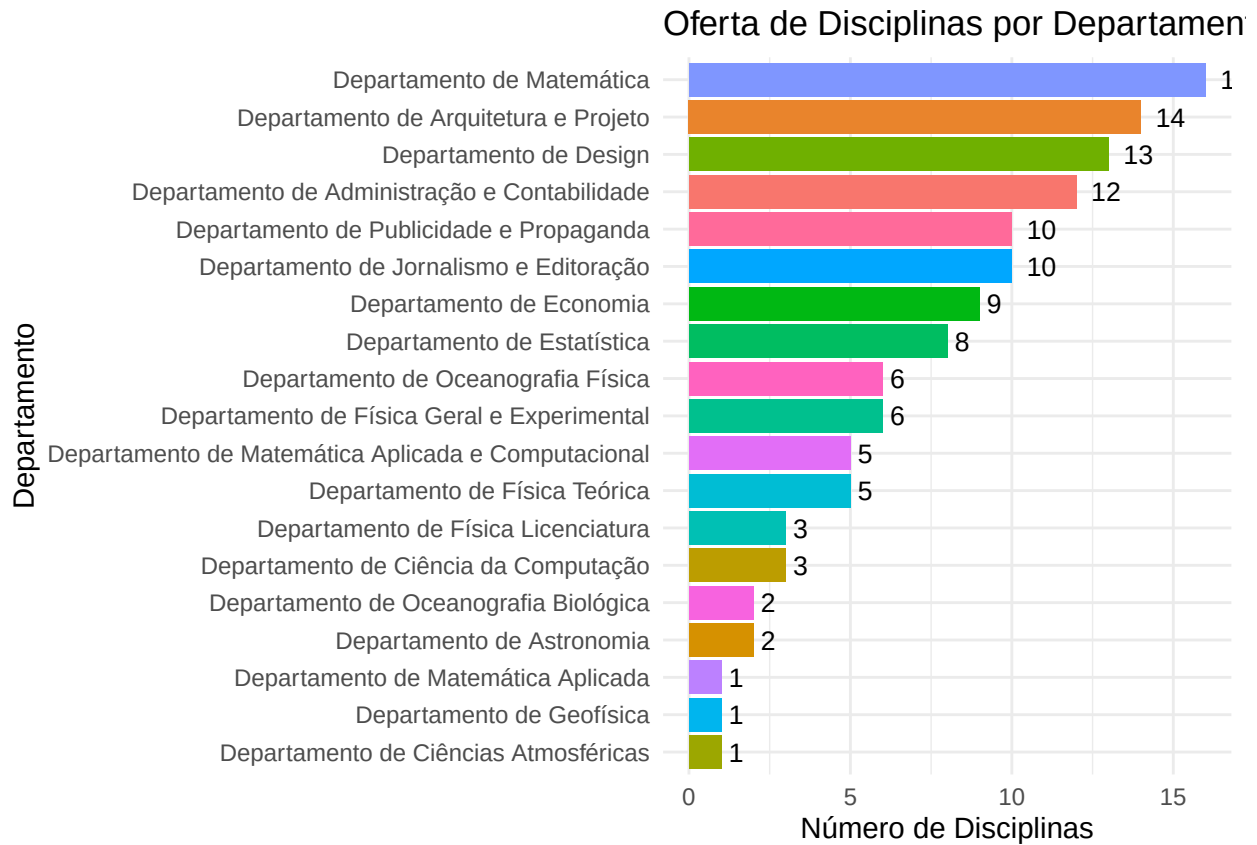
    y = "Número de Disciplinas"
  ) +
  theme_minimal()

```

```

## Don't know how to automatically pick scale for object of type <integer64>.
## Defaulting to continuous.
## Don't know how to automatically pick scale for object of type <integer64>.
## Defaulting to continuous.

```



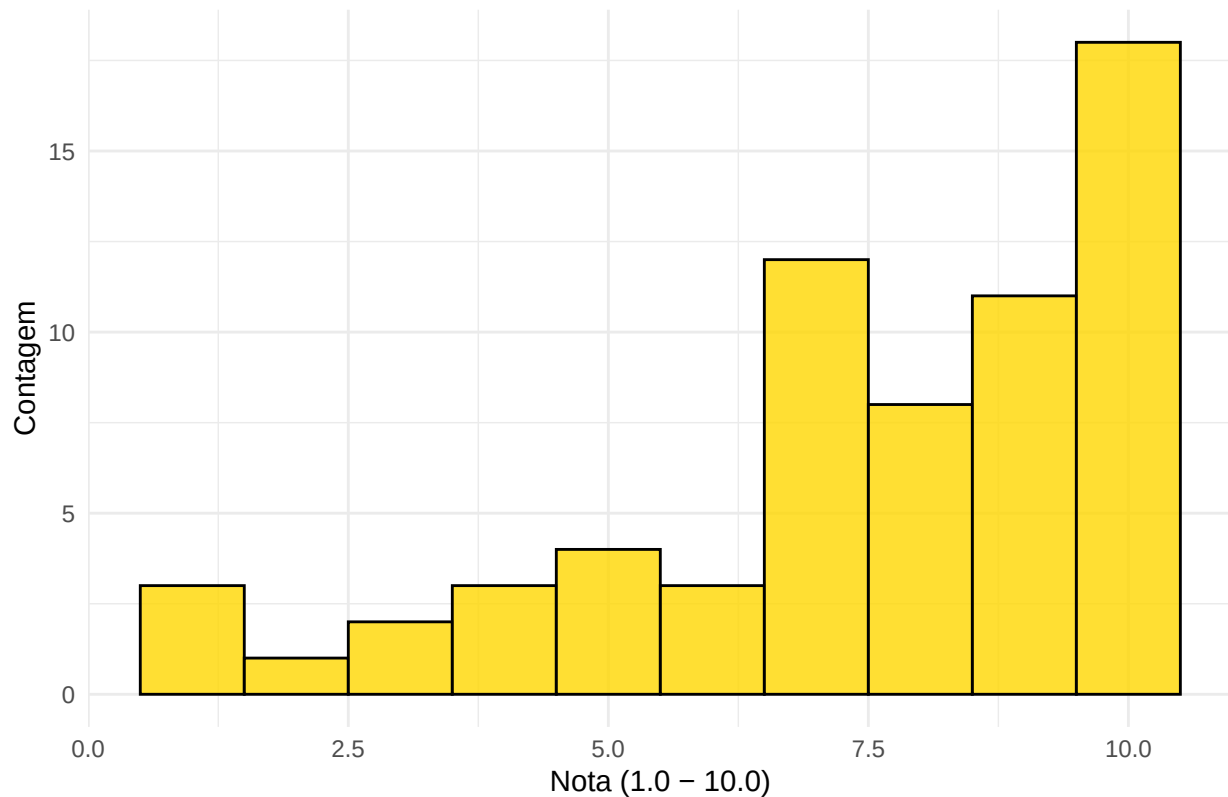
```

# Avaliações de monitores
sql_notas_monitor <- "SELECT nota FROM avaliacao_monitor;"
df_notas_monitor <- dbGetQuery(con, sql_notas_monitor)

ggplot(df_notas_monitor, aes(x = nota)) +
  geom_histogram(binwidth = 1, fill = "gold", color = "black", alpha = 0.8) +
  labs(title = "Distribuição das Notas de Avaliação de Monitores", x = "Nota (1.0 - 10.0)", y = "Cont")
  theme_minimal()

```

Distribuição das Notas de Avaliação de Monitores

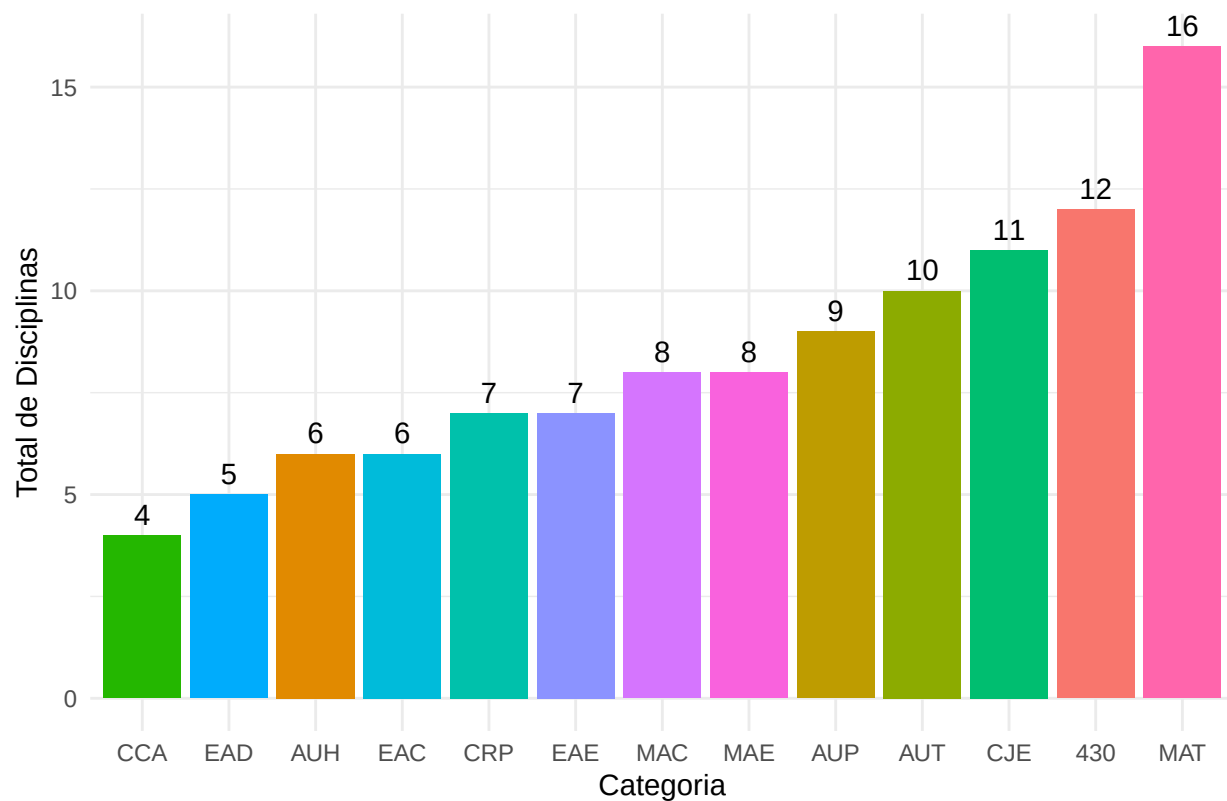


```
# Contagem de disciplinas por categoria
sql_disciplinas_categoria <- "
  SELECT
    SUBSTRING(sigla FROM 1 FOR 3) AS categoria,
    COUNT(sigla) AS total_disciplinas
  FROM
    disciplina
  GROUP BY
    categoria
  HAVING
    COUNT(sigla) >= 3 -- Filtra categorias com pelo menos 3 disciplinas
  ORDER BY
    total_disciplinas DESC;
"
df_disciplinas_categoria <- dbGetQuery(con, sql_disciplinas_categoria)

ggplot(df_disciplinas_categoria, aes(x = reorder(categoria, total_disciplinas), y = total_disciplinas,
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = total_disciplinas), vjust = -0.5) +
  labs(title = "Contagem de Disciplinas por Categoria (Prefixo)", x = "Categoria", y = "Total de Disc.
  theme_minimal()

## Don't know how to automatically pick scale for object of type <integer64>.
## Defaulting to continuous.
## Don't know how to automatically pick scale for object of type <integer64>.
## Defaulting to continuous.
```


Contagem de Disciplinas por Categoria (Prefixo)



Vínculo Trabalho

```
# Quantidade de efetivados e temporários na USP
sql_vinculo_global <- "
  SELECT
    CASE
      WHEN tipo_vinculo = 1 THEN '1 - Temporário'
      WHEN tipo_vinculo = 2 THEN '2 - Efetivo'
      ELSE 'Outro'
    END AS status,
    COUNT(nusp_associado) AS total
  FROM
    vinculo_trabalho
  GROUP BY
    tipo_vinculo, status;
"
df_vinculo_global <- dbGetQuery(con, sql_vinculo_global)

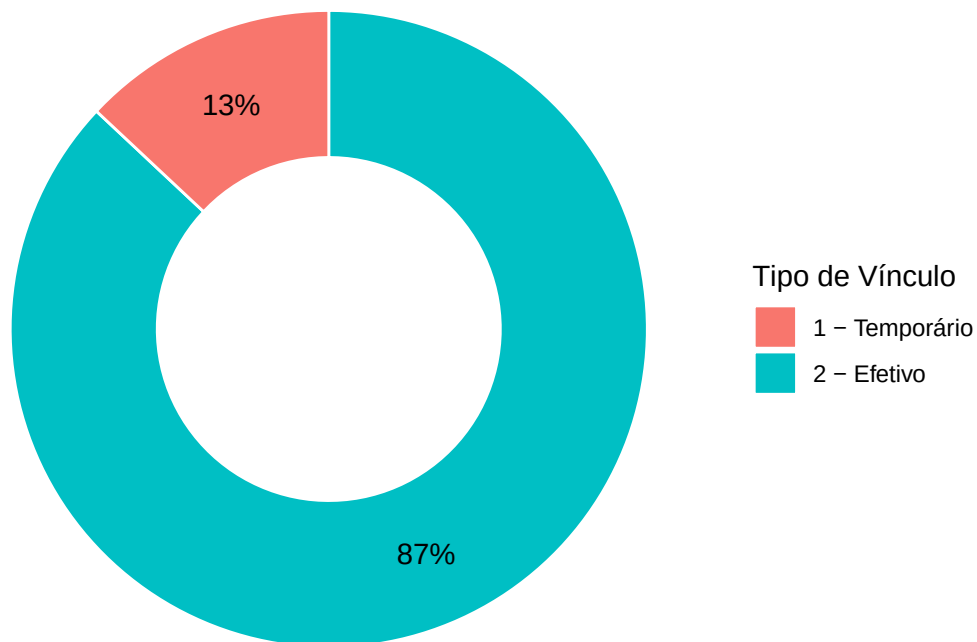
# 7. Cálculo da Porcentagem e Posição dos Rótulos
df_vinculo_global <- df_vinculo_global %>%
  mutate(
    porcentagem = total / sum(total),
    label_pos = cumsum(porcentagem) - (0.5 * porcentagem),
    label = paste0(round(porcentagem * 100), "%")
  )
```

```

)
ggplot(df_vinculo_global, aes(x = 2, y = porcentagem, fill = status)) +
  geom_bar(stat = "identity", color = "white") +
  coord_polar(theta = "y", start = 0) +
  geom_text(aes(y = label_pos, label = label), color = "black", size = 4) +
  xlim(0.5, 2.5) +
  labs(
    title = "Distribuição Global de Servidores por Tipo de Vínculo",
    fill = "Tipo de Vínculo"
  ) +
  theme_void() +
  theme(plot.title = element_text(hjust = 0.5))

```

Distribuição Global de Servidores por Tipo de Vínculo



```

#Distribuição dos efetivos e temporários por instituto
sql_vinculo_instituto <- "
  SELECT
    i.sigla AS sigla_instituto,
    CASE
      WHEN vt.tipo_vinculo = 1 THEN 'Temporário'
      WHEN vt.tipo_vinculo = 2 THEN 'Efetivo'
    END AS status_vinculo,
    COUNT(vt.nusp_associado) AS total
  FROM
    vinculo_trabalho AS vt
  JOIN

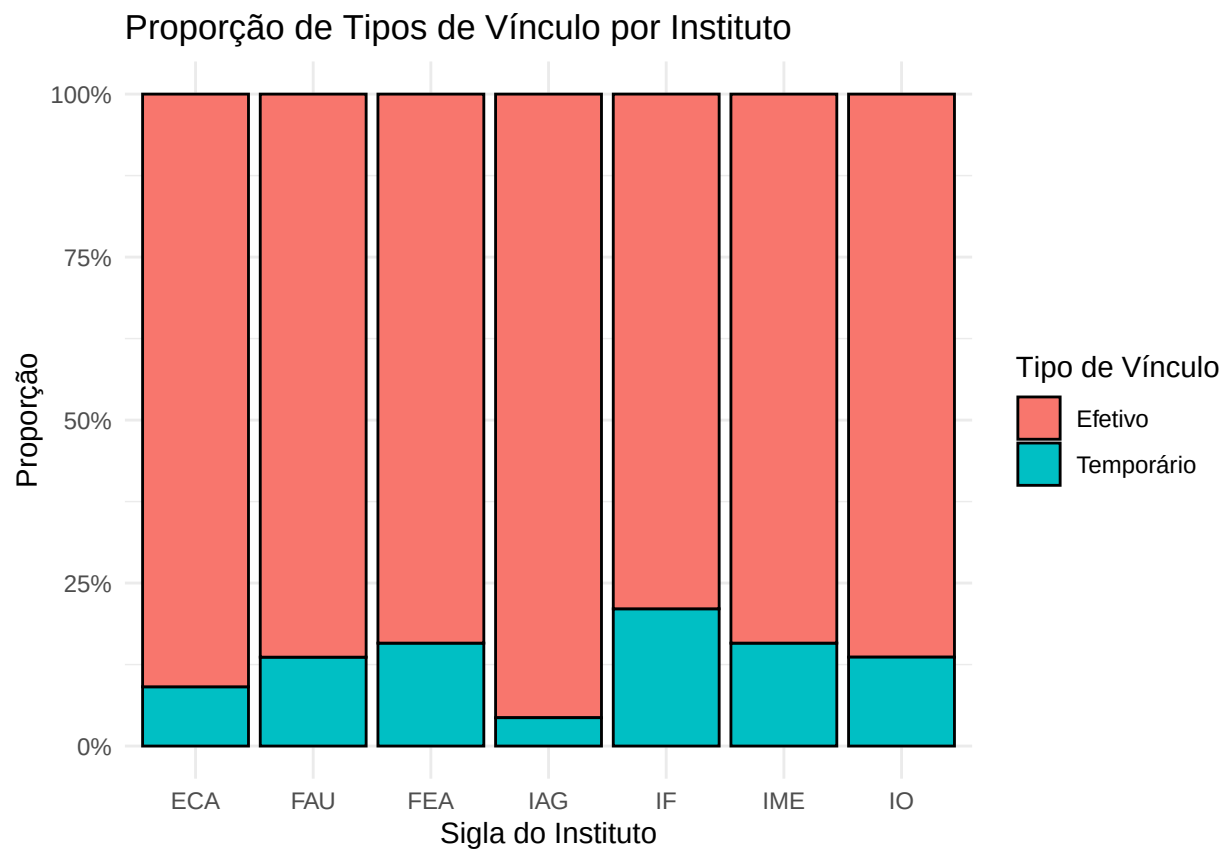
```

```

    instituto AS i ON vt.id_instituto = i.id_instituto
  GROUP BY
    i.sigla, status_vinculo;
"
df_vinculo_instituto <- dbGetQuery(con, sql_vinculo_instituto)

ggplot(df_vinculo_instituto, aes(x = sigla_instituto, y = total, fill = status_vinculo)) +
  geom_bar(stat = "identity", position = "fill", color = "black") +
  labs(
    title = "Proporção de Tipos de Vínculo por Instituto",
    x = "Sigla do Instituto",
    y = "Proporção",
    fill = "Tipo de Vínculo"
  ) +
  scale_y_continuous(labels = scales::percent) +
  theme_minimal()

```



```

# SQL: Contagem de novos vínculos (data_inicio) por ano
sql_curva_contratacoes <- "
SELECT
  EXTRACT(YEAR FROM data_inicio) AS ano_inicio,
  COUNT(nusp_associado) AS novos_vinculos
FROM
  vinculo_trabalho
GROUP BY

```

```

ano_inicio
ORDER BY
ano_inicio;
"
df_curva_contratacoes <- dbGetQuery(con, sql_curva_contratacoes)

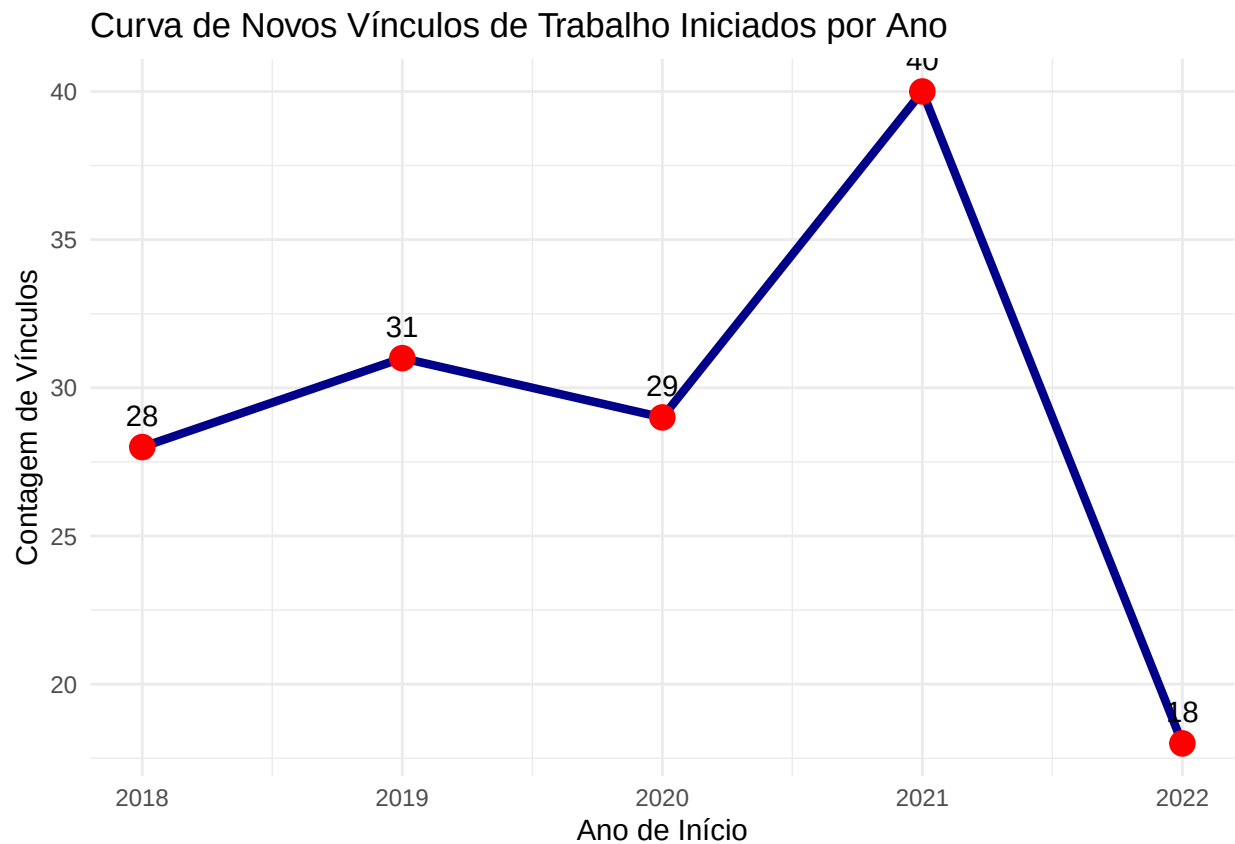
ggplot(df_curva_contratacoes, aes(x = ano_inicio, y = novos_vinculos, group = 1)) +
  geom_line(color = "darkblue", linewidth = 1.5) +
  geom_point(color = "red", size = 4) +
  geom_text(aes(label = novos_vinculos), vjust = -1, size = 4) +
  labs(
    title = "Curva de Novos Vínculos de Trabalho Iniciados por Ano",
    x = "Ano de Início",
    y = "Contagem de Vínculos"
  ) +
  theme_minimal() +
  scale_x_continuous(breaks = df_curva_contratacoes$ano_inicio)

```

```

## Don't know how to automatically pick scale for object of type <integer64>.
## Defaulting to continuous.
## Don't know how to automatically pick scale for object of type <integer64>.
## Defaulting to continuous.

```



Monitores

```

#Contagem de monitores por instituto
sql_monitores_instituto_origem <- "
  SELECT
    i.sigla AS sigla_instituto,
    COUNT(m.nusp_aluno) AS total_monitores
  FROM
    monitor AS m
  JOIN
    aluno AS a ON m.nusp_aluno = a.nusp_usuario
  JOIN
    curso AS c ON a.id_curso = c.id_curso
  JOIN
    instituto AS i ON c.id_instituto = i.id_instituto -- Assumindo que curso está ligado a instituto
  GROUP BY
    i.sigla;
"

df_monitores_instituto <- dbGetQuery(con, sql_monitores_instituto_origem)

df_monitores_instituto <- df_monitores_instituto %>%
  mutate(
    percentagem = total_monitores / sum(total_monitores),
    label_pos = cumsum(percentagem) - (0.5 * percentagem),
    label = paste0(sigla_instituto, " (", round(percentagem * 100), "%)")
  )

ggplot(df_monitores_instituto, aes(x = 2, y = percentagem, fill = sigla_instituto)) +
  geom_bar(stat = "identity", color = "white") +
  coord_polar(theta = "y", start = 0) +
  geom_text(aes(y = label_pos, label = label), color = "black", size = 3) +
  xlim(0.5, 2.5) +
  labs(title = "Proporção de Monitores por Instituto de Origem", fill = "Instituto") +
  theme_void() +
  theme(plot.title = element_text(hjust = 0.5))

```

Proporção de Monitores por Instituto de Origem

